30-Week Game Dev Challenge Schedule

One game per week, increasing difficulty, spanning genres, each built on a classic real-worl

Week	Game	Inspired By	Genre
1	Tic-Tac-Toe	Tic-Tac-Toe (traditional)	Puzzle / Abstract Strategy
2	Whack-a-Mole	Whack-a-Mole (arcade)	Reaction / Arcade
3	Pong	Pong (Atari, 1972)	Sports / Arcade
4	Snake	Snake (Nokia, 1997)	Arcade
5	Flappy Bird	Flappy Bird (2013)	Side-Scroller / Arcade
6	Breakout	Breakout (Atari, 1976)	Arcade
7	Space Invaders	Space Invaders (1978)	Shoot-'em-up
8	Astroblaster	Asteroids (Atari, 1979)	Shoot-'em-up
9	Duck Hunt	Duck Hunt (NES, 1984)	Light-Gun / Shooter
10	Memory Match	Concentration (classic card game)	Puzzle

11	Checkers	Checkers/Draughts (traditional)	Board / Abstract Strategy
12	Minesweeper	Minesweeper (Windows, 1990)	Puzzle
13	Connect Four	Connect Four (Milton Bradley, 1974)	Board / Abstract Strategy
14	Tetris	Tetris (1984)	Puzzle / Arcade
15	2048	2048 (2014)	Puzzle
16	Simon Says	Simon (Milton Bradley, 1978)	Memory / Rhythm
17	Idle Clicker	Cookie Clicker (2013)	Idle / Incremental
18	Platform Jumper	Super Mario Bros. (1985)	Platformer
19	Top-Down Adventure	The Legend of Zelda (1986)	Action-Adventure
20	Dungeon Crawler	Rogue (1980) / NetHack	Roguelike / RPG
21	Choose-Your-Own-Adventure	Zork (1977) text adventures	Interactive Fiction

22	Chess	Chess (traditional)	Board / Abstract Strategy
23	Tower Defense	Bloons TD / Fieldrunners style	Strategy
24	Match-3	Bejeweled (2001)	Puzzle
25	Lane Crosser	Frogger (1981)	Arcade
26	Maze Chase	Pac-Man (1980)	Arcade / Maze
27	Raycaster Shooter	Wolfenstein 3D (1992)	First-Person Shooter
28	Voxel World Builder	Minecraft (2011)	Sandbox / Voxel
29	Top-Down Racer	Micro Machines (1991)	Racing
30	Capstone: Mini Metroidvania	Metroid (1986) / Castlevania	Action-Adventure / RPG

'd game.

Description
Basic 2-player Tic-Tac-Toe on a 3x3 board. No AI.
Moles spawn randomly in one of three holes. Hit the matching button before it hides.
2-player Pong with paddles and a bouncing ball. No AI.
Grid-based world with a snake and food. Snake grows by one cell per food eaten; game ends if it eats itself.
Side-scrolling game where the player taps to keep a bird airborne between pipe gaps.
Bounce a ball with a paddle to break rows of bricks. Add power-ups as a stretch goal.
Defend against descending rows of aliens that shoot back, using a movable cannon.
Pilot a ship that shoots and avoids asteroids, which split into smaller pieces when hit.
Click/tap moving ducks on screen within a time limit before they escape.
Flip pairs of face-down cards to find matches; track moves and time.

Full checkers ruleset including forced jumps and kinging, 2-player local.
Reveal a grid of tiles without hitting a mine, using number clues to deduce safe cells.
Drop tokens into columns to connect four in a row; add a simple AI opponent.
Falling tetromino pieces that rotate and lock into a grid; clear full lines for points.
Slide and merge numbered tiles on a grid to reach the 2048 tile.
Repeat an increasingly long sequence of color/sound cues shown by the game.
Click to earn currency, then buy upgrades that passively generate more currency over time.
Side-scrolling platformer with running, jumping, tile-based level, and simple enemies.
Top-down tile-based world with rooms/screens, an item, and simple NPC interaction.
Turn-based dungeon crawl with procedurally generated levels, combat, and item pickups.
Text-based branching story where player choices determine the path and ending.

Full chess ruleset with legal move generation, check/checkmate detection, and a basic AI opponent.
Place towers along a path to stop waves of enemies from reaching the base.
Swap adjacent gems to form matches of 3+, triggering cascades and chain reactions.
Cross lanes of traffic and floating logs on a river to reach the goal safely.
Navigate a maze eating pellets while avoiding (or, with power pellets, hunting) ghosts.
Pseudo-3D corridor shooter rendered from a 2D map using raycasting.
Explore and edit a simple 3D world made of voxel blocks, with chunk-based terrain.
Top-down racing game with AI opponents, laps, and checkpoint-based track logic.
Explorable interconnected map with unlockable abilities gating progress, combat, and persistent save state.

Core Concepts	Difficulty (1-10)
Basic player I/O handling; CLI rendering; state machines (game states: playing/won/draw)	1
Randomness in games; timers; score-combo systems; basic event handling	1
Input-output handling; controller/keyboard mapping; collision detection & response; velocity tuning; timing/game-loop management	2
Grid-based movement; item spawning; queues/linked-lists (tail-following); how data structures shape game logic	2
Rendering vs. sprites vs. CLI (first move to true 2D sprite rendering); basic physics (gravity, impulse); collision detection; procedural obstacle spawning	2
Collision response refinement; simple level/layout data; sprite-sheet basics; power-up event triggers	3
Event systems (spawn/fire/hit events); enemy formation movement; projectile pooling; difficulty scaling over waves	3
Vector-based movement & rotation; screen wrapping; physics (thrust, drag, momentum); object pooling	3
Mouse/pointer input handling; hit-box detection; sprite animation states; controller mapping (mouse vs gamepad)	3
Model-View-Controller introduction (game state vs. rendered board vs. input handler); UI state management; simple win-condition checks	3

Turn-based game logic; move-validation rules; board representation (2D arrays); simple 'legal moves' generation	4
2D array/grid logic; flood-fill algorithm; event-driven UI (click/flag); win/lose state detection	4
Stack-based column logic; win-condition scanning (rows/cols/diagonals); intro to minimax for simple AI	4
Piece rotation systems (e.g. SRS-lite); collision & lock-delay; line-clearing algorithms; game-loop/tick timing and increasing speed curves	5
Grid-merge algorithms; animation tweening/interpolation; deeper MVC separation (model updates independent of view animation)	5
Event systems (input-vs-playback event queues); audio-visual feedback synchronization; pattern/state memory	5
Event-driven programming; save/load & persistence systems; numeric scaling/balancing; UI data-binding (MVC in practice)	5
Sprite sheets & frame animation; tile maps; platformer physics (jump arcs, gravity, coyote time); camera follow	6
Camera/room transition systems; tile-based world streaming; dialogue trees & NPC interaction; inventory basics	6
Procedural level generation; turn-based combat systems; inventory & equipment systems; permadeath state management	6
Text parsing/command interpreters; branching narrative as a state machine/graph; save-state systems	6

Full board representation & notation (FEN/PGN concepts); minimax with alpha-beta pruning; move-generation performance	7
Pathfinding (A*); wave/spawn event queues; targeting priority systems; resource/economy balancing	7
Grid-swap validation; cascade/chain resolution algorithms; particle/juice effects; combo scoring systems	7
Multi-lane collision systems; timing-based procedural obstacle generation; difficulty ramping	7
Maze/graph representation; ghost AI via finite-state machines; pathfinding on graphs; mode-timers (scatter/chase)	8
Rendering vs. sprites vs. CLI (pseudo-3D transition); raycasting fundamentals; texture mapping; performance-conscious rendering loops	8
Voxels vs. polygon meshes; chunk loading & meshing; 3D coordinate systems & camera controls; basic terrain generation (noise)	9
Vehicle physics (acceleration, friction, drift); waypoint-based AI pathing; camera framing for multiple actors; lap/checkpoint state tracking	9
Full MVC architecture integration; ability-gated world/graph design; save/load systems; combining rendering, physics, AI, and event systems from prior weeks	10

Resources
https://en.wikipedia.org/wiki/Tic-tac-toe | https://playtictactoe.org/
https://en.wikipedia.org/wiki/Whac-A-Mole
https://en.wikipedia.org/wiki/Pong | https://www.ponggame.org/
https://en.wikipedia.org/wiki/Snake_(video_game_genre)
https://en.wikipedia.org/wiki/Flappy_Bird
https://en.wikipedia.org/wiki/Breakout_(video_game)
https://en.wikipedia.org/wiki/Space_Invaders
https://en.wikipedia.org/wiki/Asteroids_(video_game)
https://en.wikipedia.org/wiki/Duck_Hunt
https://en.wikipedia.org/wiki/Concentration_(card_game)

https://en.wikipedia.org/wiki/Draughts
https://en.wikipedia.org/wiki/Minesweeper_(video_game)
https://en.wikipedia.org/wiki/Connect_Four
https://en.wikipedia.org/wiki/Tetris
https://en.wikipedia.org/wiki/2048_(video_game)
https://en.wikipedia.org/wiki/Simon_(game)
https://en.wikipedia.org/wiki/Cookie_Clicker
https://en.wikipedia.org/wiki/Super_Mario_Bros.
https://en.wikipedia.org/wiki/The_Legend_of_Zelda_(video_game)
https://en.wikipedia.org/wiki/Rogue_(video_game)
https://en.wikipedia.org/wiki/Interactive_fiction | https://en.wikipedia.org/wiki/Zork

https://en.wikipedia.org/wiki/Chess | https://en.wikipedia.org/wiki/Minimax
https://en.wikipedia.org/wiki/Tower_defense | https://en.wikipedia.org/wiki/A*_search_algorithm
https://en.wikipedia.org/wiki/Bejeweled
https://en.wikipedia.org/wiki/Frogger
https://en.wikipedia.org/wiki/Pac-Man | https://en.wikipedia.org/wiki/Finite-state_machine
https://en.wikipedia.org/wiki/Wolfenstein_3D | https://lodev.org/cgtutor/raycasting.html
https://en.wikipedia.org/wiki/Voxel | https://en.wikipedia.org/wiki/Minecraft
https://en.wikipedia.org/wiki/Micro_Machines_(video_game)
https://en.wikipedia.org/wiki/Metroidvania